LAPORAN PROJEK AKHIR

DASAR DASAR PEROGRAMAN

SISTEM RESERVASI GYM/FITNESS



Disusun Oleh:

KELOMPOK 7

Alfian Nugraha	2609116084
Novitasari Muhammad Nor	2609116082
Marexcel Elibino	2609116087
Muhammad Ardhika Prsanjayu	2609116108

Asisten Laboratorium:

Muhammd Dylan Al Furqon — **2509116038**

Muhammad Ibrahim Kamil — **2509116012**

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS MULAWARMAN

2026

KATA PENGANTAR

Puji syukur penyusun panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya penyusun dapat menyelesaikan Laporan Proyek Akhir mata kuliah Dasar Dasar Pemrograman dengan judul "Sistem Reservasi Gym & Fitness". Laporan ini disusun sebagai pertanggungjawaban atas program yang telah dibuat sekaligus memenuhi salah satu syarat penilaian mata kuliah Dasar Dasar Pemrograman.

Penyusun mengucapkan terima kasih kepada dosen pengampu, asisten laboratorium, serta rekan-rekan satu kelompok yang telah memberikan bimbingan dan bantuan selama proses pengerjaan proyek. Penyusun menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat penyusun harapkan. Semoga laporan ini bermanfaat bagi pembaca.

Samarinda, 19 Oktober 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR ..... **ii**
DAFTAR ISI ..... **iii**
DAFTAR GAMBAR ..... **iv**
DAFTAR TABEL ..... **v**
DAFTAR LAMPIRAN ..... **vi**
BAB I PENDAHULUAN **1**
1.1. Deskripsi Masalah 1
1.2. Rumusan Masalah 1
1.3. Batasan Masalah 2
1.4. Tujuan 2
1.5. Manfaat 2
BAB II PERANCANGAN **3**
2.1. Analisis Program 3
2.2. *Flowchart* 5
BAB III HASIL DAN PEMBAHASAN **8**
3.1. Implementasi Program 8
3.2. Alur Program 8
3.3. *Source Code* 16
LAMPIRAN **23**

DAFTAR GAMBAR

Gambar 2.1 *Flowchart* Reservasi Gym & Fitness (*Main*) 5
Gambar 2.2 *Flowchart* Menu *Admin* 6
Gambar 2.3 *Flowchart* Menu *Users* 6
Gambar 3.1 Tampilan Menu Utama 8
Gambar 3.2 Tampilan Registrasi 9
Gambar 3.3 Tampilan *Login* 9
Gambar 3.4 Tampilan Menu *Admin* dan Daftar Kelas 10
Gambar 3.5 Tampilan Tambah Kelas 10
Gambar 3.6 Tampilan Ubah Kelas 11
Gambar 3.7 Tampilan Hapus Kelas 11
Gambar 3.8 Tampilan Daftar Reservasi (*Admin*) 12
Gambar 3.9 Tampilan Daftar *Member* (*Admin*) 12
Gambar 3.10 Tampilan Menu *Member* 12
Gambar 3.11 Tampilan *Top Up* Saldo 13
Gambar 3.12 Tampilan Reservasi Kelas dan *Invoice* 14
Gambar 3.13 Tampilan Riwayat dan Pembatalan Reservasi 15

DAFTAR TABEL

Tabel 2.1 *Library* yang Digunakan..... 3

Tabel 2.2 Struktur Data Program..... 3

Tabel 2.3 Daftar Fungsi Program 4

Tabel 3.1 *Source Code*..... 16

DAFTAR LAMPIRAN

Lampiran 1 : Tabel Kontribusi 23

BAB I
PENDAHULUAN

1.1. Deskripsi Masalah

Pusat kebugaran (*gym*) umumnya menawarkan berbagai kelas olahraga, misalnya Zumba, Body Pump, Body Combat, Spinning, dan Pilates. Setiap kelas memiliki instruktur, jadwal, harga, dan kuota peserta yang berbeda. Keberagaman ini membuat pengelola harus mengatur banyak informasi sekaligus agar kegiatan kelas dapat berjalan dengan tertib dan nyaman bagi seluruh anggota. Semakin banyak kelas dan anggota yang dikelola, semakin besar pula beban kerja pengelola dalam menjaga data tetap akurat dan mudah diakses.

Pada banyak tempat kebugaran berskala kecil, data tersebut masih dicatat secara manual menggunakan buku atau pesan singkat. Cara ini menimbulkan beberapa masalah, yaitu data mudah hilang atau salah catat, perubahan jadwal dan harga sulit diinformasikan, dan pengelola kesulitan membedakan hak akses antara pengelola dan anggota. Selain itu, pencatatan manual membuat pengelola sulit memantau sisa kuota kelas sehingga jumlah peserta berisiko melebihi batas yang tersedia. Pencarian data tertentu, seperti daftar peserta suatu kelas, juga memakan waktu lebih lama karena pengelola harus membuka catatan satu per satu. Di sisi lain, anggota sering tidak mendapat kabar ketika jadwal atau harga berubah, sehingga dapat menimbulkan kebingungan dan menurunkan kepuasan terhadap layanan. Tanpa pembagian hak akses yang jelas, data milik pengelola juga berisiko diubah oleh pihak yang tidak berwenang.

Berdasarkan masalah tersebut, dibutuhkan sebuah program yang dapat menyimpan data kelas dan data anggota secara terstruktur, serta membedakan akses antara *admin* sebagai pengelola dan *member* sebagai pengguna layanan. Dengan program ini, *admin* dapat menambah, mengubah, dan menghapus data kelas maupun anggota dengan lebih rapi, cepat, dan minim kesalahan, sedangkan *member* dapat melihat informasi kelas yang tersedia tanpa harus bertanya langsung kepada pengelola. Program ini diharapkan dapat meningkatkan ketertiban pengelolaan data, mempermudah penyampaian informasi, serta meningkatkan kualitas pelayanan pusat kebugaran.

1.2. Rumusan Masalah

1. Bagaimana merancang program Sistem Reservasi Gym & Fitness berbasis Python yang dapat mengelola data kelas dan data anggota?
2. Bagaimana membuat sistem registrasi dan *login* yang membedakan hak akses *admin* dan *member*?
3. Bagaimana menerapkan operasi CRUD (*Create*, *Read*, *Update*, *Delete*) pada data kelas *gym*?
4. Bagaimana menyimpan data secara permanen agar tidak hilang ketika program ditutup?

1.3. Batasan Masalah

1. Program berjalan pada antarmuka teks (*terminal*/*command line*) dan dibuat menggunakan bahasa pemrograman Python.
2. Data disimpan pada *file* JSON (*users*.json dan *kelas.json*), tanpa basis data khusus.
3. Terdapat dua peran pengguna, yaitu *admin* dan *member*.
4. *Admin* dapat melihat, menambah, mengubah, dan menghapus data kelas, serta melihat daftar *member*. *Member* dapat melihat daftar kelas.
5. Registrasi hanya dapat dilakukan untuk peran *member*. Akun *admin* disediakan secara bawaan oleh sistem.

1.4. Tujuan

1. Membuat program Sistem Reservasi Gym & Fitness yang dapat mengelola data kelas dan anggota.
2. Menerapkan konsep dasar pemrograman seperti variabel, percabangan, perulangan, fungsi, struktur data *list* dan *dictionary*, serta penanganan kesalahan.
3. Menerapkan penyimpanan data permanen menggunakan *file* JSON.
4. Menerapkan sistem registrasi, *login*, dan pembagian hak akses berdasarkan peran.

1.5. Manfaat

1. Bagi penyusun: menambah pemahaman dan pengalaman menerapkan konsep dasar pemrograman pada kasus nyata.
2. Bagi pengelola *gym*: mempermudah pencatatan dan pengelolaan kelas serta data anggota.
3. Bagi *member*: memudahkan melihat informasi kelas, instruktur, jadwal, harga, dan kuota yang tersedia.
4. Bagi pembaca: menjadi referensi sederhana pembuatan aplikasi CRUD dengan penyimpanan JSON.

BAB II

PERANCANGAN

2.1. Analisis Program

Untuk membuat program ini diperlukan bahasa pemrograman Python beserta beberapa *library*. *Library* yang digunakan dijelaskan pada tabel berikut.

Tabel 2.1 *Library* yang Digunakan

Library	**Jenis**	**Kegunaan dalam Program**
json	Bawaan Python	Membaca dan menyimpan data pengguna, kelas, dan reservasi ke *file* JSON.
os	Bawaan Python	Membersihkan layar *terminal* (*cls* pada Windows, *clear* pada Linux/macOS).
time	Bawaan Python	Mencatat waktu registrasi dan reservasi serta membentuk tanggal pada nomor *invoice* dengan *time.strftime*.
random	Bawaan Python	Menghasilkan empat angka acak (*random.randint*) untuk bagian akhir nomor *invoice*.
prettytable	Pihak ketiga	Menampilkan data kelas, reservasi, dan daftar *member* dalam bentuk tabel yang rapi.
pwinput	Pihak ketiga	Menyembunyikan *password* yang diketik dan menggantinya dengan tanda bintang.

Library pihak ketiga dipasang dengan perintah *pip install prettytable pwinput*. Data program disimpan pada struktur data berikut.

Tabel 2.2 Struktur Data Program

Data	**Struktur**	**Isi**
users	*Dictionary*, kunci berupa *username*	*password*, *role* (*admin*/*user*), saldo, dan waktu terdaftar.
daftar_kelas	*List* berisi *dictionary*	id, nama, instruktur, jadwal, harga, dan kuota (sisa kuota).
reservasi	*List* berisi *dictionary*	*invoice*, *username*, *id_kelas*, *nama_kelas*, jadwal, harga, waktu, dan status (Aktif/Dibatalkan).
DEFAULT_USERS	*Dictionary*	Data bawaan berisi akun *admin*.

Data	Struktur	Isi
DEFAULT_KELAS	*List*	Lima kelas bawaan: Zumba, Body Pump, Body Combat, Spinning, dan Pilates.
PAKET_TOPUP	*Tuple*	Pilihan nominal *top up*: Rp50.000, Rp100.000, Rp200.000, dan Rp500.000.
MIN_TOPUP, MAX_TOPUP	*Integer*	Batas nominal *top up*, yaitu Rp10.000 sampai Rp5.000.000.
ROLE_VALID	*Tuple*	Daftar peran yang dikenal sistem: *admin* dan *user*.
FILE_USERS, FILE_KELAS, FILE_RESERVASI	*String*	Nama *file* JSON tempat data disimpan: *users*.json, *kelas.json*, dan *reservasi.json*.

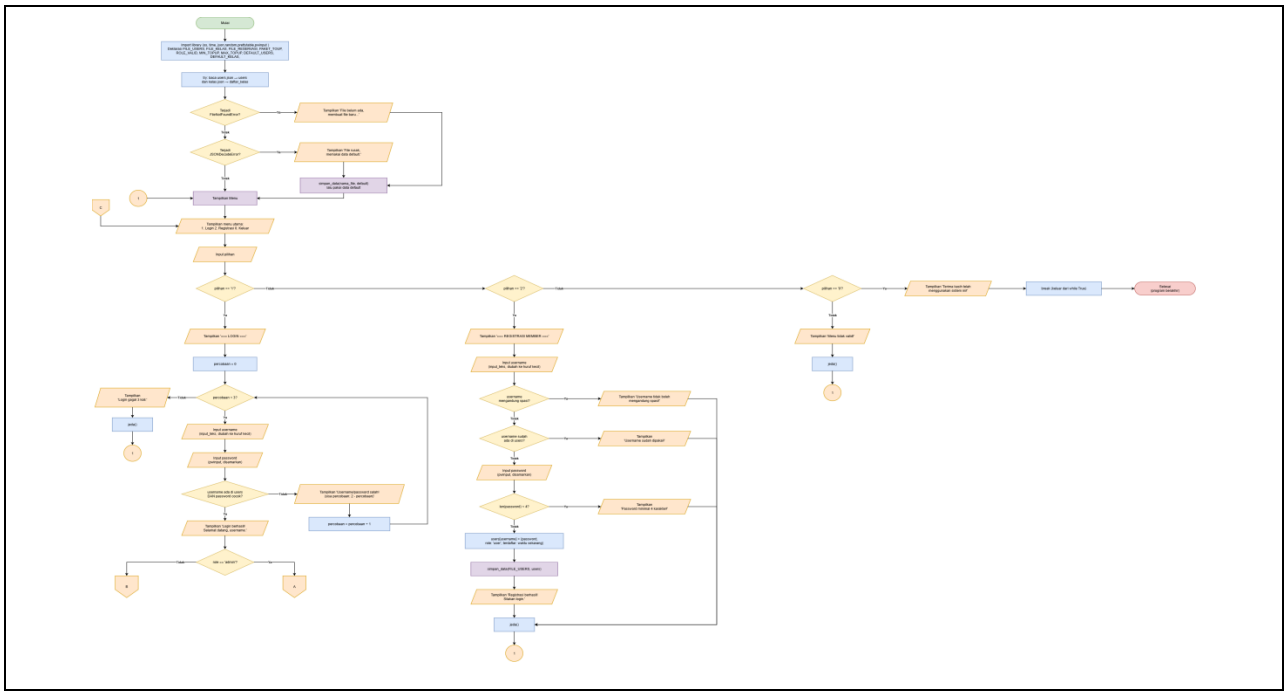
Program disusun secara modular menggunakan fungsi. Fungsi-fungsi tersebut dikelompokkan sebagai berikut.

Tabel 2.3 Daftar Fungsi Program

Kelompok	Fungsi	Keterangan
Utilitas tampilan	*bersihkan_layar, jeda, rupiah*	Membersihkan layar, menjeda program, dan memformat angka menjadi mata uang rupiah.
Validasi *input*	*input_teks, input_angka, input_password*	Memastikan *input* tidak kosong, berupa bilangan bulat sesuai batas minimum, dan *password* tersembunyi.
Penyimpanan data	*simpan_data, muat_data, lengkapi_data_user*	Menulis dan membaca *file* JSON, membuat *file* dari data bawaan jika belum ada atau rusak, serta menambah kolom saldo dan terdaftar pada *users*.json versi lama.
Tampilan data	*tampil_kelas, tampil_reservasi, tampil_users*	Menampilkan data kelas, reservasi, dan *member* dalam bentuk tabel.
Pencarian dan pengecekan	*cari_kelas, buat_id_baru, jumlah_reservasi_aktif, sudah_reservasi*	Mencari kelas berdasarkan ID, membuat ID baru otomatis, menghitung reservasi aktif suatu kelas, dan memeriksa apakah *member* sudah memesan kelas yang sama.
Autentikasi	*registrasi, login*	Mendaftarkan *member* baru dan memverifikasi pengguna dengan batas tiga kali percobaan.
CRUD kelas (*admin*)	*tambah_kelas, ubah_kelas, hapus_kelas*	Menambah, mengubah, dan menghapus data kelas.

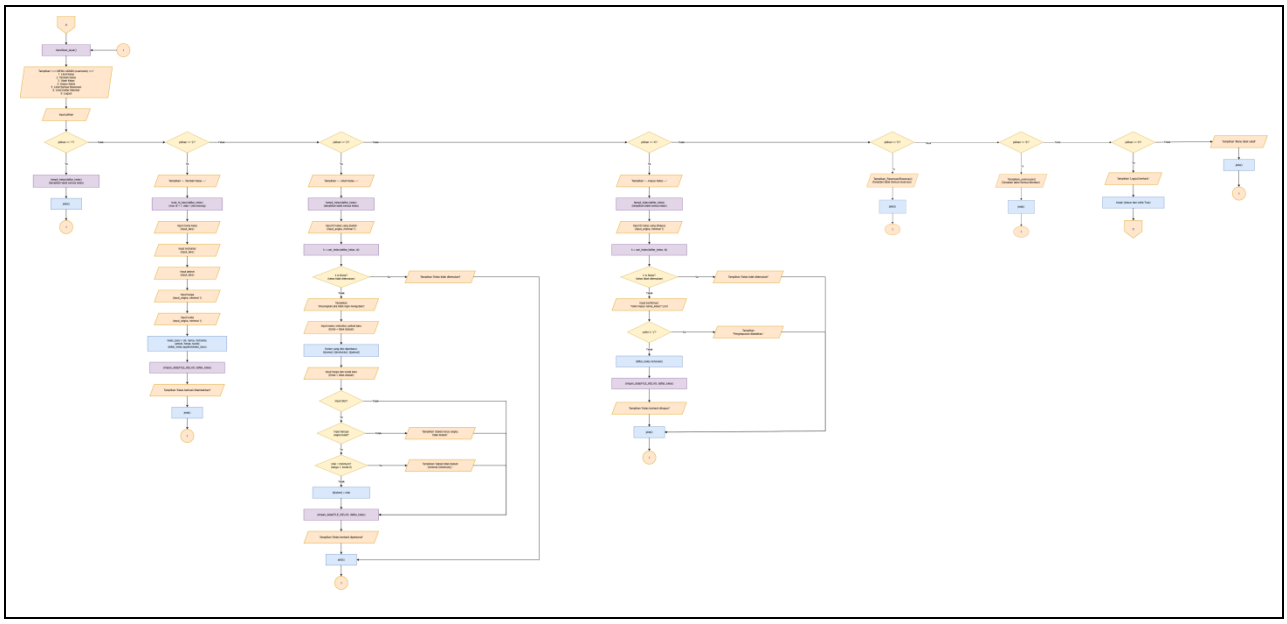
Kelompok	Fungsi	Keterangan
Fitur *member*	*top_up, reservasi_kelas, riwayat_user, batal_reservasi*	Mengisi saldo, memesan kelas, melihat riwayat reservasi, dan membatalkan reservasi dengan pengembalian saldo.
Transaksi dan *invoice*	*buat_no_invoice, bayar_saldo, cetak_invoice*	Membuat nomor *invoice*, mengurangi saldo saat pembayaran, dan mencetak bukti reservasi.
Menu dan program utama	*menu_admin, menu_user, main*	Menu untuk *admin*, menu untuk *member*, dan menu utama program.

2.2. *Flowchart*



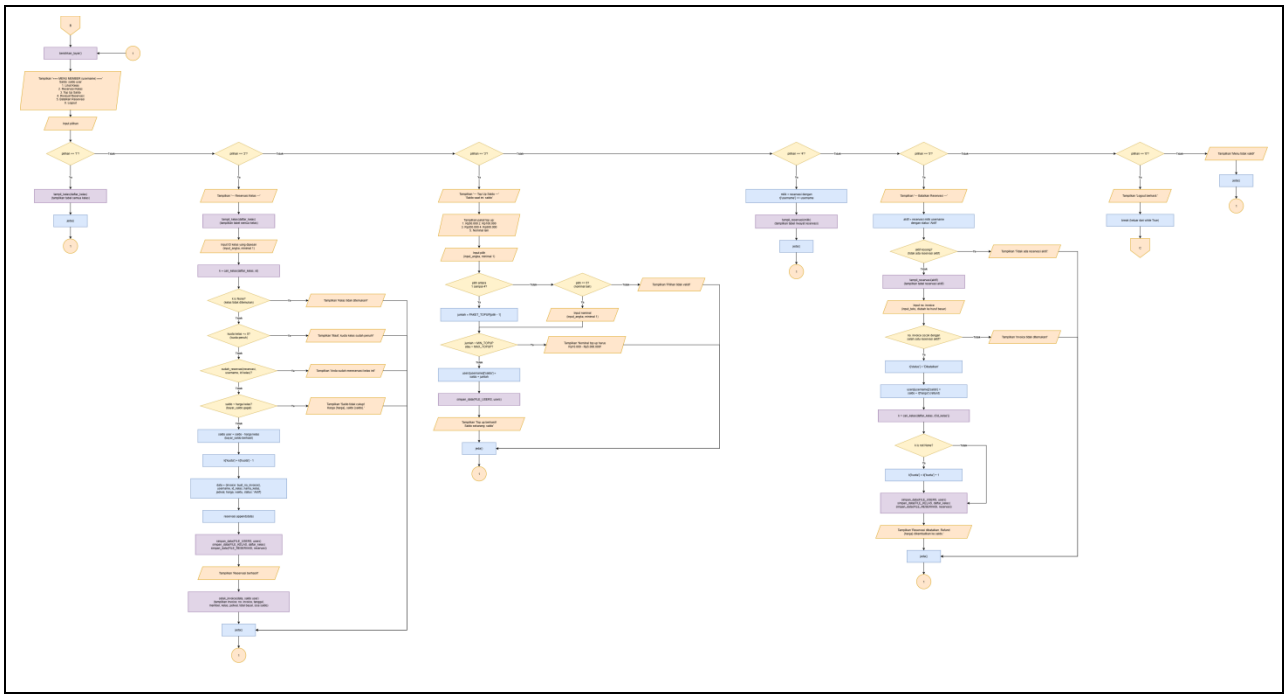
Gambar 2.1 *Flowchart* Reservasi Gym & Fitness (*Main*)

Gambar 2.1 menunjukkan alur utama program. Program dimulai dengan memuat *users*.json, *kelas.json*, dan *reservasi.json*. Jika *file* belum ada atau rusak, *file* dibuat dari data bawaan, dan akun lama dilengkapi kolom saldo dan terdaftar. Setelah itu tampil menu utama berisi *Login*, Registrasi, dan Keluar. Pada pilihan Registrasi, pengguna memasukkan *username* dan *password* yang divalidasi (tidak kosong, tanpa spasi, belum dipakai, dan *password* minimal empat karakter), lalu data disimpan ke *users*.json dan program kembali ke menu utama. Pada pilihan *Login*, pengguna diberi maksimal tiga kali percobaan; jika gagal tiga kali, program kembali ke menu utama. Jika *login* berhasil, program memeriksa *role*: *role admin* diarahkan ke Menu *Admin*, sedangkan *role user* diarahkan ke Menu *Member*. Pilihan Keluar mengakhiri program.



Gambar 2.2 *Flowchart* Menu *Admin*

Gambar 2.2 menunjukkan alur Menu *Admin*. Menu ini berisi pilihan lihat kelas, tambah kelas, ubah kelas, hapus kelas (ditolak jika kelas masih memiliki reservasi aktif), lihat semua reservasi, lihat daftar *member*, dan *logout*. Pilihan *logout* mengembalikan *admin* ke menu utama.



Gambar 2.3 *Flowchart* Menu *Users*

Gambar 2.3 menunjukkan alur Menu *Member*, yang berisi pilihan lihat kelas, reservasi kelas, *top up* saldo, riwayat reservasi, batalkan reservasi, dan *logout*. Pada *top up*, *member* memilih paket nominal atau nominal lain, program memeriksa batas Rp10.000 sampai Rp5.000.000,

menambahkannya ke saldo, lalu menyimpannya ke *users*.json. Pada reservasi kelas, *member* memilih ID kelas, lalu program memeriksa berurutan apakah kelas ada, kuota masih tersedia, *member* belum memesan kelas yang sama, dan saldo cukup. Jika semua syarat terpenuhi, saldo dan kuota dikurangi, reservasi dicatat dengan nomor *invoice*, data disimpan, dan *invoice* dicetak; jika salah satu syarat gagal, tampil pesan dan program kembali ke menu. Pada batalkan reservasi, *member* memilih nomor *invoice* yang masih aktif, status diubah menjadi Dibatalkan, saldo dikembalikan sebesar harga kelas, kuota kelas ditambah kembali, lalu data disimpan. Pilihan *logout* mengembalikan *member* ke menu utama.

BAB III
HASIL DAN PEMBAHASAN

3.1. Implementasi Program

Dalam kehidupan nyata, program ini dapat dipakai oleh pengelola pusat kebugaran berskala kecil sebagai alat bantu pengelolaan dan reservasi kelas. *Admin* (pengelola) memasukkan dan memperbarui jadwal kelas, instruktur, harga, dan kuota sesuai kondisi terbaru, serta memantau seluruh reservasi dan daftar *member*. *Member* yang telah mendaftar dapat melihat kelas yang tersedia, mengisi saldo melalui *top up*, memesan kelas, lalu menerima *invoice* sebagai bukti pembayaran. Kuota kelas berkurang otomatis setiap ada reservasi sehingga jumlah peserta tidak melebihi kapasitas.

Penyimpanan data pada tiga *file* JSON (*users*.json, *kelas.json*, dan *reservasi.json*) membuat data tetap tersedia walaupun program ditutup. Saldo *member*, sisa kuota kelas, dan riwayat reservasi ikut tersimpan sehingga pencatatan tidak lagi bergantung pada buku atau pesan manual. Ketika reservasi dibatalkan, saldo dan kuota dikembalikan sehingga catatan keuangan dan kapasitas kelas tetap konsisten. Program ini dapat dikembangkan lebih lanjut, misalnya dengan pengamanan *password* dan antarmuka grafis, sebagaimana dijelaskan pada saran.

3.2. Alur Program

```
print("   SISTEM RESERVASI GYM & FITNESS")
print("=" * 36)
print("1. Login")
print("2. Registrasi")
print("0. Keluar")
pilihan = input("Pilih menu: ").strip()
```

Gambar 3.1 Tampilan Menu Utama

Kode tersebut digunakan untuk menampilkan menu utama sistem reservasi gym dan fitness. Pengguna dapat memilih menu Login untuk masuk ke akun, Registrasi untuk membuat akun baru, atau Keluar untuk mengakhiri program. Perintah input() digunakan untuk menerima pilihan pengguna, sedangkan .strip() menghapus spasi di awal dan akhir input agar pilihan lebih mudah divalidasi.

```python
def registrasi(users):
    print("\n=== REGISTRASI MEMBER ===")
    username = input_teks("Username baru : ").lower()
    if " " in username:
        print("Username tidak boleh mengandung spasi!")
        return
    if username in users:
        print("Username sudah dipakai!")
        return
    password = input_password("Password      : ")
    if len(password) < 4:
        print("Password minimal 4 karakter!")
        return
```

Gambar 3.2 Tampilan Registrasi

Bagian registrasi digunakan untuk membuat akun member baru dengan memasukkan username dan password. Program memeriksa agar username tidak mengandung spasi, belum digunakan, dan password memiliki minimal empat karakter. Sementara itu, bagian login digunakan untuk memeriksa kecocokan username dan password. Pengguna diberikan maksimal tiga kali percobaan untuk masuk ke dalam sistem.

```python
def login(users):
    """Return: username jika berhasil, None jika gagal 3x."""
    print("\n=== LOGIN ===")
    for percobaan in range(3):
        username = input_teks("Username : ").lower()
        password = input_password("Password : ")
        if username in users and users[username]["password"] == password:
            print(f"Login berhasil! Selamat datang, {username}.")
            return username
        print(f"Username/password salah! (sisa percobaan: {2 - percobaan})")
    print("Login gagal 3 kali.")
    return None
```

Gambar 3.3 Tampilan *Login*

Kode login digunakan untuk memeriksa identitas pengguna sebelum memberikan akses ke dalam sistem. Program meminta username dan password, kemudian mencocokkannya dengan data akun yang tersimpan di dalam variabel users. Perulangan for percobaan in range(3) memberikan kesempatan kepada pengguna untuk mencoba login sebanyak tiga kali.

```python
def menu_admin(username, users, daftar_kelas, reservasi):
    while True:
        bersihkan_layar()
        print(f"=== MENU ADMIN ({username}) ===")
        print("1. Lihat Kelas")
        print("2. Tambah Kelas")
        print("3. Ubah Kelas")
        print("4. Hapus Kelas")
        print("5. Lihat Semua Reservasi")
        print("6. Lihat Daftar Member")
        print("0. Logout")
        pilihan = input("Pilih menu: ").strip()
```

Gambar 3.4 Tampilan Menu *Admin* dan Daftar Kelas

Kode ini digunakan untuk menampilkan menu khusus admin setelah berhasil login menggunakan akun dengan peran admin. Perulangan while True membuat menu terus ditampilkan sampai admin memilih menu Logout. Fungsi bersihkan_layar() digunakan untuk membersihkan tampilan terminal sebelum menu ditampilkan kembali sehingga tampilan lebih mudah dibaca.

Menu admin menyediakan enam fitur utama, yaitu melihat daftar kelas, menambah kelas, mengubah kelas, menghapus kelas, melihat semua reservasi, dan melihat daftar member.

```python
def tambah_kelas(daftar_kelas):
    print("\n--- Tambah Kelas ---")
    kelas_baru = {
        "id": buat_id_baru(daftar_kelas),
        "nama": input_teks("Nama kelas  : "),
        "instruktur": input_teks("Instruktur  : "),
        "jadwal": input_teks("Jadwal      : "),
        "harga": input_angka("Harga       : ", 1),
        "kuota": input_angka("Kuota       : ", 1),
    }
    daftar_kelas.append(kelas_baru)
    simpan_data(FILE_KELAS, daftar_kelas)
    print("Kelas berhasil ditambahkan!")
```

Gambar 3.5 Tampilan Tambah Kelas

Kode tersebut digunakan untuk menambahkan data kelas baru ke dalam sistem reservasi gym dan fitness. Ketika admin memilih menu tambah kelas, program meminta beberapa informasi, yaitu nama kelas, instruktur, jadwal, harga, dan kuota. Fungsi buat_id_baru() digunakan untuk menghasilkan ID kelas secara otomatis agar setiap kelas memiliki identitas yang berbeda.

```python
def ubah_kelas(daftar_kelas):
    print("\n--- Ubah Kelas ---")
    tampil_kelas(daftar_kelas)
    k = cari_kelas(daftar_kelas, input_angka("ID kelas yang diubah: ", 1))
    if k is None:
        print("Kelas tidak ditemukan!")
        return
    print("(Kosongkan jika tidak ingin mengubah)")
    for kolom, label in (("nama", "Nama"), ("instruktur", "Instruktur"), ("jadwal", "Jadwal")):
        baru = input(f"{label} [{k[kolom]}]: ").strip()
        if baru:
            k[kolom] = baru
    for kolom, label, minimum in (("harga", "Harga", 1), ("kuota", "Kuota", 0)):
        baru = input(f"{label} [{k[kolom]}]: ").strip()
        if baru:
            try:
                nilai = int(baru)
                if nilai < minimum:
                    print(f"{label} tidak diubah (minimal {minimum}).")
                else:
                    k[kolom] = nilai
            except ValueError:
                print(f"{label} harus angka, tidak diubah.")
    simpan_data(FILE_KELAS, daftar_kelas)
    print("Kelas berhasil diperbarui!")
```

Gambar 3.6 Tampilan Ubah Kelas

Kode ini digunakan untuk memperbarui informasi kelas yang sudah terdaftar di dalam sistem. Pertama, program menampilkan daftar kelas agar admin dapat melihat ID kelas yang ingin diubah. Setelah itu, admin memasukkan ID kelas yang dipilih. Fungsi cari_kelas() digunakan untuk mencari data kelas berdasarkan ID tersebut.

```python
def hapus_kelas(daftar_kelas, reservasi):
    print("\n--- Hapus Kelas ---")
    tampil_kelas(daftar_kelas)
    k = cari_kelas(daftar_kelas, input_angka("ID kelas yang dihapus: ", 1))
    if k is None:
        print("Kelas tidak ditemukan!")
        return
    if jumlah_reservasi_aktif(reservasi, k["id"]) > 0:
        print("Kelas masih memiliki reservasi aktif, tidak bisa dihapus!")
        return
    if input(f"Yakin hapus '{k['nama']}'? (y/n): ").lower() == "y":
        daftar_kelas.remove(k)
        simpan_data(FILE_KELAS, daftar_kelas)
        print("Kelas berhasil dihapus!")
    else:
        print("Penghapusan dibatalkan.")
```

Gambar 3.7 Tampilan Hapus Kelas

Kode ini berfungsi untuk menghapus data kelas yang tidak lagi diperlukan oleh admin. Program terlebih dahulu menampilkan daftar kelas dan meminta admin memasukkan ID kelas yang ingin dihapus. Fungsi cari_kelas() kemudian mencari kelas berdasarkan ID tersebut. Jika kelas tidak ditemukan, program menampilkan pesan bahwa kelas tidak tersedia.

```python
def tampil_reservasi(daftar):
    if not daftar:
        print("Belum ada data reservasi.")
        return
    tabel = PrettyTable(["Invoice", "Member", "Kelas", "Jadwal", "Harga", "Waktu Pesan", "Status"])
    for r in daftar:
        tabel.add_row([r["invoice"], r["username"], r["nama_kelas"], r["jadwal"],
            rupiah(r["harga"]), r["waktu"], r["status"]])
    print(tabel)
```

Gambar 3.8 Tampilan Daftar Reservasi (*Admin*)

Kode tersebut digunakan untuk menampilkan data reservasi yang tersimpan di dalam sistem. Program terlebih dahulu memeriksa apakah terdapat data reservasi. Jika daftar masih kosong, program menampilkan pesan bahwa belum ada data reservasi. Namun, jika data tersedia, program membuat tabel menggunakan PrettyTable dengan beberapa kolom, yaitu nomor invoice, username member, nama kelas, jadwal, harga, waktu pemesanan, dan status reservasi.

```python
def tampil_users(users):
    tabel = PrettyTable(["Username", "Role", "Saldo", "Terdaftar"])
    for nama, data in users.items():
        tabel.add_row([nama, data["role"], rupiah(data["saldo"]), data["terdaftar"]])
    print(tabel)
```

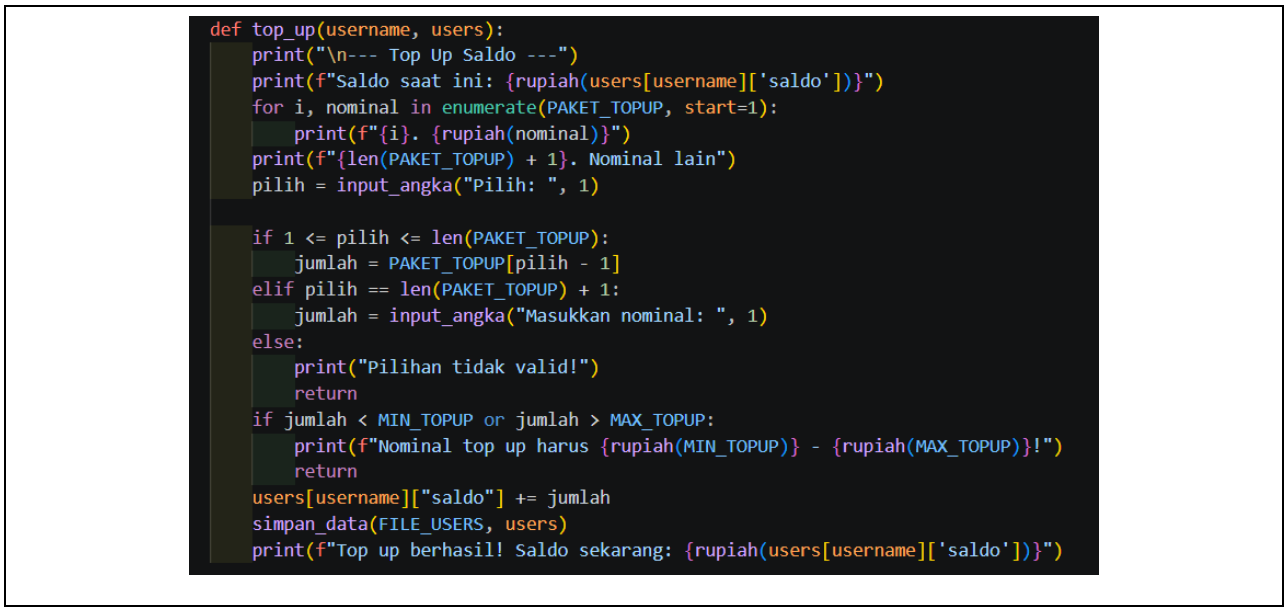
Gambar 3.9 Tampilan Daftar *Member* (*Admin*)

Kode yang digunakan adalah fungsi tampil_users(users). Fungsi ini menampilkan daftar akun pengguna dalam bentuk tabel yang berisi username, role, saldo, dan tanggal terdaftar. Fitur ini membantu admin melihat data pengguna yang tersimpan di dalam sistem.

```python
def menu_user(username, users, daftar_kelas, reservasi):
    while True:
        bersihkan_layar()
        print(f"=== MENU MEMBER ({username}) ===")
        print(f"Saldo: {rupiah(users[username]['saldo'])}")
        print("1. Lihat Kelas")
        print("2. Reservasi Kelas")
        print("3. Top Up Saldo")
        print("4. Riwayat Reservasi")
        print("5. Batalkan Reservasi")
        print("0. Logout")
        pilihan = input("Pilih menu: ").strip()
```

Gambar 3.10 Tampilan Menu *Member*

Kode yang digunakan berfungsi untuk menampilkan menu *member* yang dibuat oleh fungsi *menu_user()*. Fungsi ini menampilkan saldo *member* pada setiap tampilan menu dan berisi pilihan lihat kelas, reservasi kelas, *top up* saldo, riwayat reservasi, serta batalkan reservasi, ditambah *logout*.



```python
def top_up(username, users):
    print("\n--- Top Up Saldo ---")
    print(f"Saldo saat ini: {rupiah(users[username]['saldo'])}")
    for i, nominal in enumerate(PAKET_TOPUP, start=1):
        print(f"{i}. {rupiah(nominal)}")
    print(f"{len(PAKET_TOPUP) + 1}. Nominal lain")
    pilih = input_angka("Pilih: ", 1)

    if 1 <= pilih <= len(PAKET_TOPUP):
        jumlah = PAKET_TOPUP[pilih - 1]
    elif pilih == len(PAKET_TOPUP) + 1:
        jumlah = input_angka("Masukkan nominal: ", 1)
    else:
        print("Pilihan tidak valid!")
        return
    if jumlah < MIN_TOPUP or jumlah > MAX_TOPUP:
        print(f"Nominal top up harus {rupiah(MIN_TOPUP)} - {rupiah(MAX_TOPUP)}!")
        return
    users[username]["saldo"] += jumlah
    simpan_data(FILE_USERS, users)
    print(f"Top up berhasil! Saldo sekarang: {rupiah(users[username]['saldo'])}")
```

Gambar 3.11 Tampilan *Top Up* Saldo

Kode yang digunakan adalah fungsi top_up(username, users). Fungsi ini memungkinkan member menambahkan saldo dengan memilih nominal yang tersedia atau memasukkan nominal sendiri. Program memeriksa apakah nominal sesuai batas minimal Rp10.000 dan maksimal Rp5.000.000 sebelum saldo ditambahkan dan disimpan.

```python
def reservasi_kelas(username, users, daftar_kelas, reservasi):
    print("\n--- Reservasi Kelas ---")
    tampil_kelas(daftar_kelas)
    k = cari_kelas(daftar_kelas, input_angka("ID kelas yang dipesan: ", 1))
    if k is None:
        print("Kelas tidak ditemukan!")
        return
    if k["kuota"] <= 0:
        print("Maaf, kuota kelas sudah penuh!")
        return
    if sudah_reservasi(reservasi, username, k["id"]):
        print("Anda sudah mereservasi kelas ini!")
        return
    if not bayar_saldo(users, username, k["harga"]):
        print(f"Saldo tidak cukup! Harga {rupiah(k['harga'])}, saldo {rupiah(users[username]['saldo'])}.")
        return

    k["kuota"] -= 1
    data = {
        "invoice": buat_no_invoice(),
        "username": username,
        "id_kelas": k["id"],
        "nama_kelas": k["nama"],
        "jadwal": k["jadwal"],
        "harga": k["harga"],
        "waktu": time.strftime("%d-%m-%Y %H:%M"),
        "status": "Aktif",
    }
    reservasi.append(data)
    simpan_data(FILE_USERS, users)
    simpan_data(FILE_KELAS, daftar_kelas)
    simpan_data(FILE_RESERVASI, reservasi)
    print("Reservasi berhasil!")
    cetak_invoice(data, users[username]["saldo"])
```

Gambar 3.12 Tampilan Reservasi Kelas dan *Invoice*

Kode yang digunakan adalah fungsi reservasi_kelas(username, users, daftar_kelas, reservasi) dan cetak_invoice(r, sisa_saldo). Fungsi reservasi memeriksa ketersediaan kelas, kuota, reservasi ganda, dan kecukupan saldo. Jika reservasi berhasil, saldo member dan kuota kelas diperbarui, lalu invoice ditampilkan berisi nomor invoice, tanggal, nama member, kelas, jadwal, total pembayaran, dan sisa saldo.

```python
def riwayat_user(username, reservasi):
    """Return: list reservasi milik user."""
    milik = [r for r in reservasi if r["username"] == username]
    tampil_reservasi(milik)
    return milik
```

```python
def batal_reservasi(username, users, daftar_kelas, reservasi):
    print("\n--- Batalkan Reservasi ---")
    aktif = [r for r in reservasi if r["username"] == username and r["status"] == "Aktif"]
    if not aktif:
        print("Tidak ada reservasi aktif.")
        return
    tampil_reservasi(aktif)
    no = input_teks("Masukkan no. invoice: ").upper()
    for r in aktif:
        if r["invoice"] == no:
            r["status"] = "Dibatalkan"
            users[username]["saldo"] += r["harga"]
            k = cari_kelas(daftar_kelas, r["id_kelas"])
            if k is not None:
                k["kuota"] += 1
            simpan_data(FILE_USERS, users)
            simpan_data(FILE_KELAS, daftar_kelas)
            simpan_data(FILE_RESERVASI, reservasi)
            print(f"Reservasi dibatalkan. Refund {rupiah(r['harga'])} dikembalikan ke saldo.")
            return
    print("Invoice tidak ditemukan!")
```

Gambar 3.13 Tampilan Riwayat dan Pembatalan Reservasi

Kode ini menjelaskan fitur riwayat dan pembatalan reservasi pada sistem gym dan fitness. Pada fitur riwayat reservasi, program menggunakan fungsi riwayat_user() untuk menampilkan daftar reservasi yang dimiliki oleh member berdasarkan username yang sedang login. Data reservasi tersebut ditampilkan dalam bentuk tabel yang berisi nomor invoice, nama member, nama kelas, jadwal, harga, waktu pemesanan, dan status reservasi.

Selain itu, terdapat fitur pembatalan reservasi melalui fungsi batal_reservasi(). Member dapat memilih reservasi yang masih berstatus aktif dengan memasukkan nomor invoice. Jika invoice ditemukan, status reservasi akan diubah menjadi Dibatalkan, saldo member akan dikembalikan sesuai harga kelas, dan kuota kelas akan bertambah kembali. Setelah itu, perubahan data saldo, kelas, dan reservasi disimpan ke dalam sistem. Jika tidak ada reservasi aktif atau nomor invoice tidak ditemukan, program akan menampilkan pesan yang sesuai.

3.3. *Source Code*

Tabel 3.1 *Source Code*

```
import json
import os
import time
import random
from prettytable import PrettyTable
import pwinput

FILE_USERS = "users.json"
FILE_KELAS = "kelas.json"
FILE_RESERVASI = "reservasi.json"

PAKET_TOPUP = (50000, 100000, 200000, 500000)
ROLE_VALID = ("admin", "user")
MIN_TOPUP = 10000
MAX_TOPUP = 5000000

DEFAULT_USERS = {
    "admin": {"password": "admin123", "role": "admin", "saldo": 0, "terdaftar": "-"},
}
DEFAULT_KELAS = [
    {"id": 1, "nama": "Zumba", "instruktur": "Fina", "jadwal": "Senin 07:00", "harga": 50000,
        "kuota": 15},
    {"id": 2, "nama": "Body Pump", "instruktur": "Juhdi", "jadwal": "Selasa 17:00", "harga":
        40000, "kuota": 20},
    {"id": 3, "nama": "Body Combat", "instruktur": "Alfian", "jadwal": "Rabu 18:30", "harga":
        60000, "kuota": 12},
    {"id": 4, "nama": "Spinning", "instruktur": "Excel", "jadwal": "Kamis 06:30", "harga":
        55000, "kuota": 10},
    {"id": 5, "nama": "Pilates", "instruktur": "Vita", "jadwal": "Jumat 16:00", "harga":
        65000, "kuota": 8},
]


def bersihkan_layar():
    os.system("cls" if os.name == "nt" else "clear")

def jeda():
    input("\nTekan Enter untuk melanjutkan...")

def rupiah(angka):
    return "Rp{:,}".format(angka).replace(",", ".")

def input_teks(pesan):
    """Input teks tidak boleh kosong. Return: string."""
    while True:
        nilai = input(pesan).strip()
        if nilai:
            return nilai
        print("Input tidak boleh kosong!")

def input_angka(pesan, minimum=0):
    """Input bilangan bulat >= minimum. Return: int."""
    while True:
        try:
            nilai = int(input(pesan))
            if nilai < minimum:
                print(f"Nilai minimal {minimum}!")
                continue
            return nilai
        except ValueError:
            print("Input harus berupa angka bulat!")

def input_password(pesan):
```

```
    """Input password tersembunyi (pwinput). Return: string."""
    while True:
        pw = pwinput.pwinput(pesan, mask="*").strip()
        if pw:
            return pw
        print("Password tidak boleh kosong!")


def simpan_data(nama_file, data):
    try:
        with open(nama_file, "w", encoding="utf-8") as f:
            json.dump(data, f, indent=4, ensure_ascii=False)
    except OSError:
        print(f"Gagal menyimpan ke {nama_file}!")

def muat_data(nama_file, default):
    """Muat JSON. Jika file belum ada/rusak, buat dari data default."""
    try:
        with open(nama_file, "r", encoding="utf-8") as f:
            return json.load(f)
    except FileNotFoundError:
        print(f"File {nama_file} belum ada, membuat file baru...")
        simpan_data(nama_file, default)
        return json.loads(json.dumps(default))
    except json.JSONDecodeError:
        print(f"File {nama_file} rusak, memakai data default.")
        simpan_data(nama_file, default)
        return json.loads(json.dumps(default))

def lengkapi_data_user(users):
    """Menambah kolom saldo/terdaftar pada users.json versi lama. Return: users."""
    for data in users.values():
        data.setdefault("saldo", 0)
        data.setdefault("terdaftar", "-")
    return users


def tampil_kelas(daftar_kelas):
    if not daftar_kelas:
        print("Belum ada data kelas.")
        return
    tabel = PrettyTable(["ID", "Nama Kelas", "Instruktur", "Jadwal", "Harga", "Sisa Kuota"])
    for k in daftar_kelas:
        tabel.add_row([k["id"], k["nama"], k["instruktur"], k["jadwal"], rupiah(k["harga"]),
            k["kuota"]])
    print(tabel)

def tampil_reservasi(daftar):
    if not daftar:
        print("Belum ada data reservasi.")
        return
    tabel = PrettyTable(["Invoice", "Member", "Kelas", "Jadwal", "Harga", "Waktu Pesan",
        "Status"])
    for r in daftar:
        tabel.add_row([r["invoice"], r["username"], r["nama_kelas"], r["jadwal"],
            rupiah(r["harga"]), r["waktu"], r["status"]])
    print(tabel)

def tampil_users(users):
    tabel = PrettyTable(["Username", "Role", "Saldo", "Terdaftar"])
    for nama, data in users.items():
        tabel.add_row([nama, data["role"], rupiah(data["saldo"]), data["terdaftar"]])
    print(tabel)

def cetak_invoice(r, sisa_saldo):
    print("\n" + "=" * 40)
    print("        INVOICE RESERVASI GYM")
    print("=" * 40)
    print(f"No. Invoice : {r['invoice']}")
```

```
    print(f"Tanggal     : {r['waktu']}")
    print(f"Member      : {r['username']}")
    print(f"Kelas       : {r['nama_kelas']}")
    print(f"Jadwal      : {r['jadwal']}")
    print(f"Total Bayar : {rupiah(r['harga'])}")
    print(f"Sisa Saldo  : {rupiah(sisa_saldo)}")
    print("=" * 40)
    print("   Terima kasih, selamat berlatih!")
    print("=" * 40)


def cari_kelas(daftar_kelas, id_kelas):
    """Return: dict kelas atau None."""
    for k in daftar_kelas:
        if k["id"] == id_kelas:
            return k
    return None

def buat_id_baru(daftar_kelas):
    if not daftar_kelas:
        return 1
    return max(k["id"] for k in daftar_kelas) + 1

def buat_no_invoice():
    return "INV-" + time.strftime("%y%m%d") + "-" + str(random.randint(1000, 9999))

def jumlah_reservasi_aktif(reservasi, id_kelas):
    total = 0
    for r in reservasi:
        if r["id_kelas"] == id_kelas and r["status"] == "Aktif":
            total += 1
    return total

def sudah_reservasi(reservasi, username, id_kelas):
    for r in reservasi:
        if r["username"] == username and r["id_kelas"] == id_kelas and r["status"] == "Aktif":
            return True
    return False

def bayar_saldo(users, username, jumlah):
    """Kurangi saldo. Return True jika berhasil, False jika saldo kurang."""
    if users[username]["saldo"] < jumlah:
        return False
    users[username]["saldo"] -= jumlah
    return True


def registrasi(users):
    print("\n=== REGISTRASI MEMBER ===")
    username = input_teks("Username baru : ").lower()
    if " " in username:
        print("Username tidak boleh mengandung spasi!")
        return
    if username in users:
        print("Username sudah dipakai!")
        return
    password = input_password("Password      : ")
    if len(password) < 4:
        print("Password minimal 4 karakter!")
        return
    users[username] = {
        "password": password,
        "role": "user",
        "saldo": 0,
        "terdaftar": time.strftime("%d-%m-%Y %H:%M"),
    }
    simpan_data(FILE_USERS, users)
    print("Registrasi berhasil! Silakan login.")
```

```
def login(users):
    """Return: username jika berhasil, None jika gagal 3x."""
    print("\n=== LOGIN ===")
    for percobaan in range(3):
        username = input_teks("Username : ").lower()
        password = input_password("Password : ")
        if username in users and users[username]["password"] == password:
            print(f"Login berhasil! Selamat datang, {username}.")
            return username
        print(f"Username/password salah! (sisa percobaan: {2 - percobaan})")
    print("Login gagal 3 kali.")
    return None


def tambah_kelas(daftar_kelas):
    print("\n--- Tambah Kelas ---")
    kelas_baru = {
        "id": buat_id_baru(daftar_kelas),
        "nama": input_teks("Nama kelas  : "),
        "instruktur": input_teks("Instruktur  : "),
        "jadwal": input_teks("Jadwal      : "),
        "harga": input_angka("Harga       : ", 1),
        "kuota": input_angka("Kuota       : ", 1),
    }
    daftar_kelas.append(kelas_baru)
    simpan_data(FILE_KELAS, daftar_kelas)
    print("Kelas berhasil ditambahkan!")

def ubah_kelas(daftar_kelas):
    print("\n--- Ubah Kelas ---")
    tampil_kelas(daftar_kelas)
    k = cari_kelas(daftar_kelas, input_angka("ID kelas yang diubah: ", 1))
    if k is None:
        print("Kelas tidak ditemukan!")
        return
    print("(Kosongkan jika tidak ingin mengubah)")
    for kolom, label in (("nama", "Nama"), ("instruktur", "Instruktur"), ("jadwal",
        "Jadwal")):
        baru = input(f"{label} [{k[kolom]}]: ").strip()
        if baru:
            k[kolom] = baru
    for kolom, label, minimum in (("harga", "Harga", 1), ("kuota", "Kuota", 0)):
        baru = input(f"{label} [{k[kolom]}]: ").strip()
        if baru:
            try:
                nilai = int(baru)
                if nilai < minimum:
                    print(f"{label} tidak diubah (minimal {minimum}).")
                else:
                    k[kolom] = nilai
            except ValueError:
                print(f"{label} harus angka, tidak diubah.")
    simpan_data(FILE_KELAS, daftar_kelas)
    print("Kelas berhasil diperbarui!")

def hapus_kelas(daftar_kelas, reservasi):
    print("\n--- Hapus Kelas ---")
    tampil_kelas(daftar_kelas)
    k = cari_kelas(daftar_kelas, input_angka("ID kelas yang dihapus: ", 1))
    if k is None:
        print("Kelas tidak ditemukan!")
        return
    if jumlah_reservasi_aktif(reservasi, k["id"]) > 0:
        print("Kelas masih memiliki reservasi aktif, tidak bisa dihapus!")
        return
    if input(f"Yakin hapus '{k['nama']}'? (y/n): ").lower() == "y":
        daftar_kelas.remove(k)
        simpan_data(FILE_KELAS, daftar_kelas)
        print("Kelas berhasil dihapus!")
```

```
    else:
        print("Penghapusan dibatalkan.")

def menu_admin(username, users, daftar_kelas, reservasi):
    while True:
        bersihkan_layar()
        print(f"=== MENU ADMIN ({username}) ===")
        print("1. Lihat Kelas")
        print("2. Tambah Kelas")
        print("3. Ubah Kelas")
        print("4. Hapus Kelas")
        print("5. Lihat Semua Reservasi")
        print("6. Lihat Daftar Member")
        print("0. Logout")
        pilihan = input("Pilih menu: ").strip()

        if pilihan == "1":
            tampil_kelas(daftar_kelas)
        elif pilihan == "2":
            tambah_kelas(daftar_kelas)
        elif pilihan == "3":
            ubah_kelas(daftar_kelas)
        elif pilihan == "4":
            hapus_kelas(daftar_kelas, reservasi)
        elif pilihan == "5":
            tampil_reservasi(reservasi)
        elif pilihan == "6":
            tampil_users(users)
        elif pilihan == "0":
            print("Logout berhasil.")
            break
        else:
            print("Menu tidak valid!")
        jeda()


def top_up(username, users):
    print("\n--- Top Up Saldo ---")
    print(f"Saldo saat ini: {rupiah(users[username]['saldo'])}")
    for i, nominal in enumerate(PAKET_TOPUP, start=1):
        print(f"{i}. {rupiah(nominal)}")
    print(f"{len(PAKET_TOPUP) + 1}. Nominal lain")
    pilih = input_angka("Pilih: ", 1)

    if 1 <= pilih <= len(PAKET_TOPUP):
        jumlah = PAKET_TOPUP[pilih - 1]
    elif pilih == len(PAKET_TOPUP) + 1:
        jumlah = input_angka("Masukkan nominal: ", 1)
    else:
        print("Pilihan tidak valid!")
        return
    if jumlah < MIN_TOPUP or jumlah > MAX_TOPUP:
        print(f"Nominal top up harus {rupiah(MIN_TOPUP)} - {rupiah(MAX_TOPUP)}!")
        return
    users[username]["saldo"] += jumlah
    simpan_data(FILE_USERS, users)
    print(f"Top up berhasil! Saldo sekarang: {rupiah(users[username]['saldo'])}")

def reservasi_kelas(username, users, daftar_kelas, reservasi):
    print("\n--- Reservasi Kelas ---")
    tampil_kelas(daftar_kelas)
    k = cari_kelas(daftar_kelas, input_angka("ID kelas yang dipesan: ", 1))
    if k is None:
        print("Kelas tidak ditemukan!")
        return
    if k["kuota"] <= 0:
        print("Maaf, kuota kelas sudah penuh!")
        return
    if sudah_reservasi(reservasi, username, k["id"]):
```

```
        print("Anda sudah mereservasi kelas ini!")
        return
    if not bayar_saldo(users, username, k["harga"]):
        print(f"Saldo tidak cukup! Harga {rupiah(k['harga'])}, saldo
            {rupiah(users[username]['saldo'])}.")
        return

    k["kuota"] -= 1
    data = {
        "invoice": buat_no_invoice(),
        "username": username,
        "id_kelas": k["id"],
        "nama_kelas": k["nama"],
        "jadwal": k["jadwal"],
        "harga": k["harga"],
        "waktu": time.strftime("%d-%m-%Y %H:%M"),
        "status": "Aktif",
    }
    reservasi.append(data)
    simpan_data(FILE_USERS, users)
    simpan_data(FILE_KELAS, daftar_kelas)
    simpan_data(FILE_RESERVASI, reservasi)
    print("Reservasi berhasil!")
    cetak_invoice(data, users[username]["saldo"])

def riwayat_user(username, reservasi):
    """Return: list reservasi milik user."""
    milik = [r for r in reservasi if r["username"] == username]
    tampil_reservasi(milik)
    return milik

def batal_reservasi(username, users, daftar_kelas, reservasi):
    print("\n--- Batalkan Reservasi ---")
    aktif = [r for r in reservasi if r["username"] == username and r["status"] == "Aktif"]
    if not aktif:
        print("Tidak ada reservasi aktif.")
        return
    tampil_reservasi(aktif)
    no = input_teks("Masukkan no. invoice: ").upper()
    for r in aktif:
        if r["invoice"] == no:
            r["status"] = "Dibatalkan"
            users[username]["saldo"] += r["harga"]
            k = cari_kelas(daftar_kelas, r["id_kelas"])
            if k is not None:
                k["kuota"] += 1
            simpan_data(FILE_USERS, users)
            simpan_data(FILE_KELAS, daftar_kelas)
            simpan_data(FILE_RESERVASI, reservasi)
            print(f"Reservasi dibatalkan. Refund {rupiah(r['harga'])} dikembalikan ke saldo.")
            return
    print("Invoice tidak ditemukan!")

def menu_user(username, users, daftar_kelas, reservasi):
    while True:
        bersihkan_layar()
        print(f"=== MENU MEMBER ({username}) ===")
        print(f"Saldo: {rupiah(users[username]['saldo'])}")
        print("1. Lihat Kelas")
        print("2. Reservasi Kelas")
        print("3. Top Up Saldo")
        print("4. Riwayat Reservasi")
        print("5. Batalkan Reservasi")
        print("0. Logout")
        pilihan = input("Pilih menu: ").strip()

        if pilihan == "1":
            tampil_kelas(daftar_kelas)
        elif pilihan == "2":
```

```
            reservasi_kelas(username, users, daftar_kelas, reservasi)
        elif pilihan == "3":
            top_up(username, users)
        elif pilihan == "4":
            riwayat_user(username, reservasi)
        elif pilihan == "5":
            batal_reservasi(username, users, daftar_kelas, reservasi)
        elif pilihan == "0":
            print("Logout berhasil.")
            break
        else:
            print("Menu tidak valid!")
        jeda()


def main():
    users = lengkapi_data_user(muat_data(FILE_USERS, DEFAULT_USERS))
    daftar_kelas = muat_data(FILE_KELAS, DEFAULT_KELAS)
    reservasi = muat_data(FILE_RESERVASI, [])

    while True:
        bersihkan_layar()
        print("=" * 36)
        print("   SISTEM RESERVASI GYM & FITNESS")
        print("=" * 36)
        print("1. Login")
        print("2. Registrasi")
        print("0. Keluar")
        pilihan = input("Pilih menu: ").strip()

        if pilihan == "1":
            username = login(users)
            if username is not None:
                if users[username]["role"] == "admin":
                    menu_admin(username, users, daftar_kelas, reservasi)
                else:
                    menu_user(username, users, daftar_kelas, reservasi)
            else:
                jeda()
        elif pilihan == "2":
            registrasi(users)
            jeda()
        elif pilihan == "0":
            print("Terima kasih telah menggunakan sistem ini!")
            break
        else:
            print("Menu tidak valid!")
            jeda()


if __name__ == "__main__":
    while True:
        try:
            main()
            break
        except KeyboardInterrupt:
            print("\nCtrl+C ditekan, kembali ke menu awal...")
            time.sleep(1)
```

LAMPIRAN

Lampiran 1 : Tabel Kontribusi

Nama	Kontribusi	Bagian
Alfian Nugraha (2609116084)	Membuat laporan dan *flowchart*	Laporan: BAB I Pendahuluan, BAB II Perancangan, dan BAB III Hasil dan Pembahasan. *Flowchart*: pada BAB II Perancangan.
Novitasari Muhammad Nor (2609116082)	Membuat laporan dan *flowchart*	Laporan: penambahan Daftar Gambar dan penomoran halaman. *Flowchart*: pada BAB II Perancangan.
Marexcel Elibino (2609116087)	Membuat *flowchart*	*Flowchart*: pada BAB II Perancangan.
Muhammad Ardhika Prsanjayu (2609116108)	Membuat kode Python dan data JSON	Kode program Python (konsul2.py) serta *file* data JSON (*users*.json, *kelas.json*, dan *reservasi.json*).